

Mobile Programming

Driving Sim

By:

- Jerell
- Faliq
- Lance
- Shafiq
- Zhun Hang
- Zhi Cheng

Table of Contents

Table of Contents	2
Overview	3
Gameplay Loop	3
Game Mechanics	4
Implemented Features	6
1. Sensors (Accelerometer) Implementation	6
2. Animations (Sprite Change) Implementation	7
3. OpenGL ES 3.0 Implementation	8
4. Camera (Perspective View)	8
5. Controls	9
Libraries/Assets	10

Overview



“Driving Sim” is an endless runner type game where the player controls a car that moves forward endlessly while attempting to avoid obstacles coming their way.

Gameplay Loop

The player controls the left and right movements of a car that is constantly moving forward. As time passes, obstacles will appear on the road and the player must avoid hitting the obstacles by moving the car left or right.

If the player hits an obstacle, the current gameplay session ends and a score will be given to the player based on the amount of distance the player managed to travel before hitting the obstacle.

Game Mechanics

Driving Sim features easy to learn mechanics.



The game starts with the player and their car driving forward indefinitely. The player will **earn points** the longer they are able to continue driving on the road without hitting any obstacles.



The player can tilt their phone to adjust the direction that the car is facing. This will enable the player to maneuver around and past obstacles that they will encounter through the game. In the above example, the phone is tilted to the right, thus turning the car towards the right.

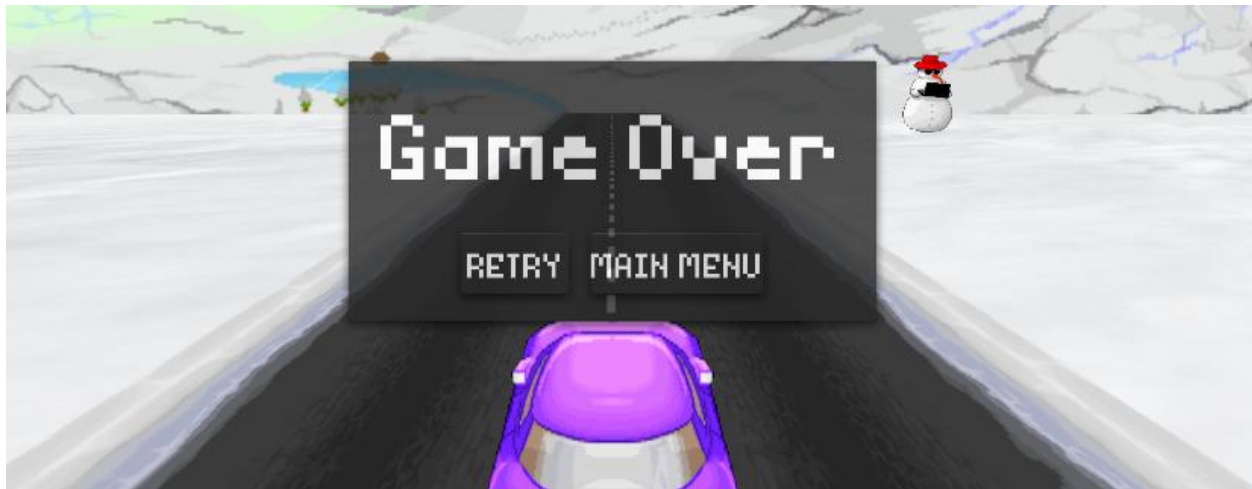


The player can tilt their phone to adjust the direction that the car is facing. This will enable the player to maneuver around and past obstacles that they will encounter through the game. In the above example, the phone is tilted to the left, thus turning the car towards the left.



The player must avoid such obstacles to continue earning more points through the game.





In the event the player fails to avoid the obstacle the game will end and the player will be presented with the option to retry or go back to the main menu.

Implemented Features

1. Sensors (Accelerometer) Implementation

The app uses the device's accelerometer to detect tilt. This is useful for controlling movement, like simulating steering a car in a racing game.

How it works in the application:

- The TiltSensor class registers for accelerometer updates using a high-frequency update rate.
- The accelerometer data is adjusted based on the device's current rotation.
- The sensor reading is scaled and clamped, before applying a decay and smoothing factor. This is to prevent abrupt changes.
- The final tilt value is then used to calculate the player's animation offset and position.

For example, if you tilt your phone to the left, the game might move the player in that direction.

2. Animations (Sprite Change) Implementation

To make objects feel more alive, the app changes their appearance based on how much the phone is tilted. This is done using sprite-based animations, which means swapping between different images (or frames) to create movement.

How it works in the application:

- Each object (like a car or character) has multiple frames of an animation.
- When the phone tilts, the app calculates how much the animation should change.
- The app selects the correct frame to display, creating the illusion of motion.

For example, if you imagine a car in a racing game, tilting left might cause it to lean in that direction, while tilting right does the opposite. This is controlled by adjusting which image of the car is shown.

3. OpenGL ES 3.0 Implementation

To create a visually rich 2.5D world, the app uses OpenGL ES 3.0, a powerful tool for rendering graphics on mobile devices.

What it does:

- Renders objects like the player, roads, and obstacles.
- Uses meshes with z-axis to allow 3D rotation making them look realistic.
- Enables depth testing to render 2D texture within 3D coordinates.
- Passes through a simple shader that handles sprite sheet animations for a more immersive experience.

How it works in the application:

- The app creates objects like roads and obstacles and assigns them a position in the world.
- The game renders the objects based on the camera's position and the player's movement.

For example, in a racing game, the road and obstacles are drawn ahead of the player, and as they move forward, new sections of the road appear to keep the scene continuous.

4. Camera (Perspective View)

The app utilizes advanced graphics techniques to render a visually engaging 2.5D world.

What it does:

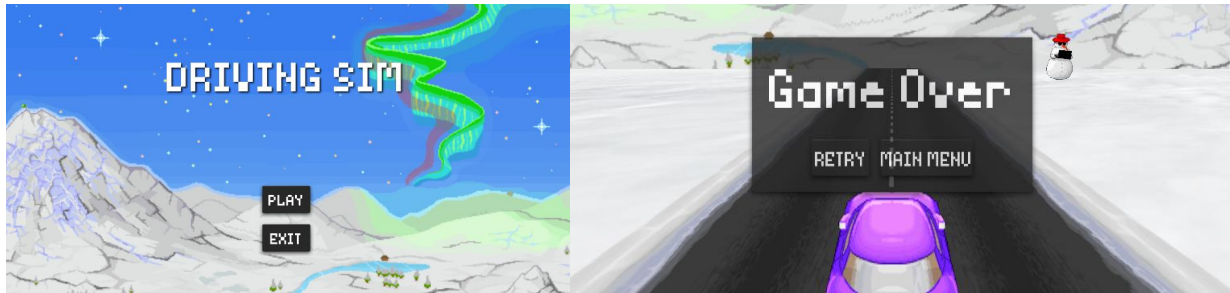
- Handles camera positioning, allowing different perspectives (e.g., top-down or first-person view).

How it works in the application:

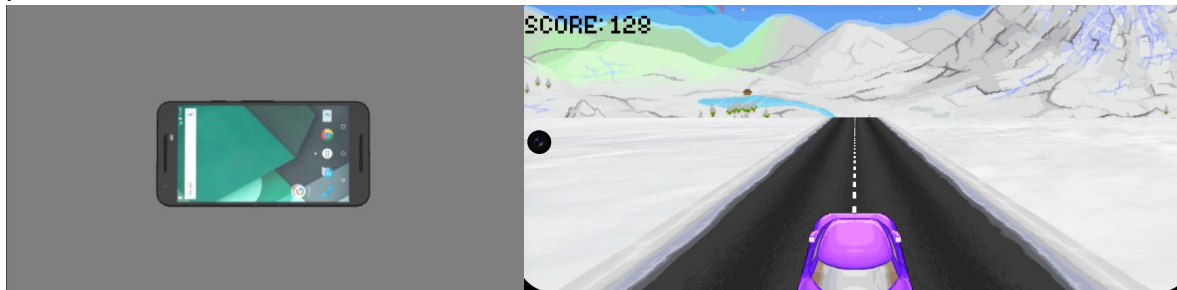
- Every object other than the player is rendered using a top down view camera. This gives the perspective of objects getting smaller the further they are from the player.
- The player is then rendered using the default camera to remain clipped to the bottom screen of the phone.
- Objects will appear bigger as it gets closer to the player.

5. Controls

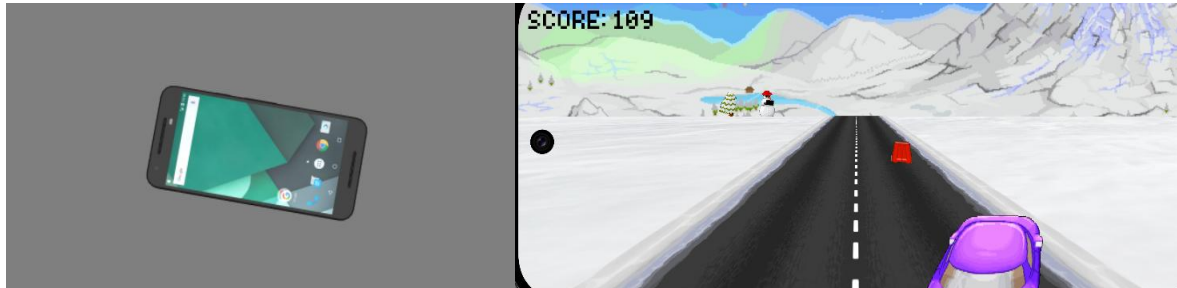
The player can tap on the screen to select the options that they want to choose in the main menu and the lose screen.



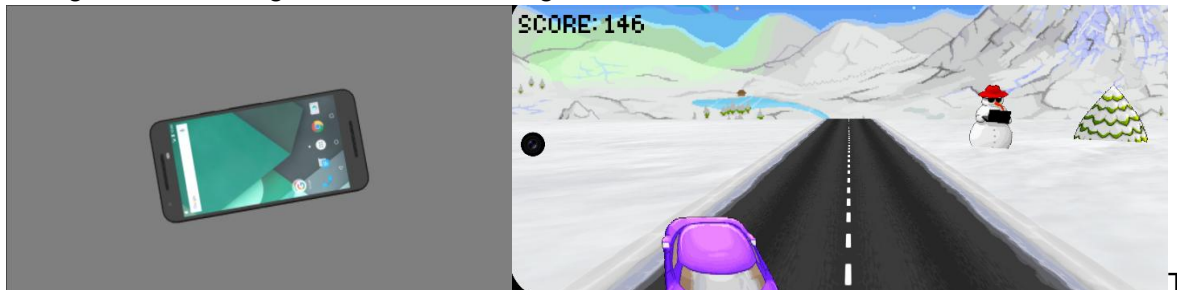
The player can turn their car left or right by tilting their phone left or right as shown in the pictures below.



With no inputs the car will go in a straight line.



Tilting towards the right will turn the car right.



Tilting

towards the left will turn the car left.

Libraries/Assets

Art:

<https://opengameart.org/content/25d-racing-resources>

<https://opengameart.org/content/pixel-font>

<https://opengameart.org/content/seamless-snow-texture-0>

Library:

- OpenGL ES 3.0
- Android sensor API
- Android UI framework
- Kotlin standard library